🚀 ArgoCD Complete Interview Mastery Guide

For DevOps / SRE / Platform Engineers

> **How to use this guide:** Every section gives you: Simple Explanation → Why It Exists → Internals → Interview Answer (Short) → Interview Answer (Deep) → Real YAML → Interview Questions → Gotchas → Connections ⚠ = Commonly misunderstood or frequently asked tricky topic

---

📋 Table of Contents

1. What is ArgoCD — The Big Picture
2. GitOps: The Philosophy Behind ArgoCD
3. ArgoCD Architecture — Deep Dive
4. Core Concepts: Applications, Projects, AppSets
5. Sync Policies, Sync Waves, and Sync Hooks
6. Health Checks and Resource Tracking
7. Drift Detection and Reconciliation Loops
8. Manifest Sources: Helm, Kustomize, Plain YAML
9. GitOps Repo Structure for Production
10. App of Apps Pattern
11. ApplicationSets — Fleet Management
12. Multi-Cluster Deployments
13. RBAC and SSO Integration
14. Secrets Management with ArgoCD
15. CI + CD Integration: Jenkins + ArgoCD
16. Argo Rollouts: Progressive Delivery
17. ArgoCD CLI, UI, and API
18. ArgoCD in Production: SRE Scenarios
19. Interview Quiz — Strict Mode

---

1. What is ArgoCD — The Big Picture

🟢 Simple Explanation

ArgoCD is a **declarative, GitOps continuous delivery tool for Kubernetes**. It watches a Git repository and ensures the Kubernetes cluster always matches what is defined in Git. If someone manually changes something in the cluster, ArgoCD detects the drift and either alerts you or automatically reverts it.

Think of it as: **Git is the source of truth. ArgoCD is the enforcer.**

🔍 Why It Exists — The Problem It Solves

Before ArgoCD (Push-based CI/CD):

```
Developer → CI Pipeline (Jenkins/GitHub Actions) → kubectl apply → Cluster
```

Problems with this:

- CI pipelines need `kubectl` access and cluster credentials — a security risk
- No auditability — who deployed what and when?
- If someone does `kubectl edit` in prod, nobody knows
- Rollback = re-running an old pipeline (fragile)
- No single source of truth for what's running

With ArgoCD (Pull-based GitOps):

```
Developer → Git Commit → ArgoCD watches → Pulls manifest → Applies to
cluster
```

ArgoCD solves:

- ✅ Cluster credentials never leave the cluster (pull model)
- ✅ Git history = full audit trail of every deployment
- ✅ Drift detection — cluster state vs Git state is always compared
- ✅ Rollback = revert a Git commit
- ✅ Self-healing — ArgoCD auto-reverts manual changes

🎯 Short Interview Answer

> *"ArgoCD is a GitOps-based continuous delivery tool for Kubernetes. It monitors a Git repository as the source of truth and continuously reconciles the cluster state to match what's declared in Git. Unlike push-based CD pipelines where CI tools run kubectl commands, ArgoCD pulls configuration from Git, which means cluster credentials never leave the cluster, every change is auditable via Git history, and drift is automatically detected and optionally self-healed."*

📡 Deeper Version

> *"ArgoCD uses a pull-based model — it runs inside the cluster and polls or receives webhook notifications from Git. It compares the live cluster state against the desired state in Git using a reconciliation loop every 3 minutes by default. If it detects a diff — called drift — it marks the application as OutOfSync. With automated sync enabled, it applies the changes. With self-heal enabled, it also reverts manual changes to the cluster. This is fundamentally different from CI-driven kubectl apply because the source of truth lives in Git, not in a pipeline."*

---

2. GitOps: The Philosophy Behind ArgoCD

🟢 What is GitOps?

GitOps is an operational framework where:

1. **Entire system state is declared in Git** (infrastructure, app configs, policies)
2. **Git is the single source of truth** — not the cluster, not a CI pipeline
3. **Changes happen via Git operations** — PRs, commits, merges
4. **Automated agents reconcile** the live state to match Git

The 4 GitOps Principles (OpenGitOps)

Principle	Meaning
Declarative	System state described as declarations, not scripts
Versioned & Immutable	Git stores desired state with history
Pulled Automatically	Software agents pull desired state from Git
Continuously Reconciled	Agents detect and correct drift automatically

⚠️ Push-Based vs Pull-Based — Common Interview Trap

Feature	Push-Based (Jenkins → kubectl)	Pull-Based (ArgoCD)
Who initiates deploy	CI pipeline pushes	ArgoCD pulls from Git
Credential risk	CI server has cluster creds	Cluster creds stay in cluster
Drift detection	None	Continuous
Audit trail	CI logs (incomplete)	Git history (complete)
Rollback	Re-run old pipeline	`git revert`
Self-healing	Not possible	Built-in

🎯 Short Interview Answer

> *"GitOps uses Git as the single source of truth for infrastructure and application state. All changes go through Git, and an automated agent — in this case ArgoCD — continuously reconciles the live cluster to match Git. The pull model means the cluster pulls its own configuration rather than a CI pipeline pushing to it, which dramatically improves security and auditability."*

---

3. ArgoCD Architecture — Deep Dive

Component Overview

```
┌──────────────────────────────────────────────────────────────────┐
│                      ArgoCD Control Plane                        │
│                                                                  │
│  ┌──────────────┐   ┌──────────────┐   ┌──────────────────┐    │
│  │ API Server   │   │ Repo Server  │   │ App Controller   │    │
│  │ (REST/gRPC)  │   │(Git clone/   │   │(Reconcile loop,  │    │
│  │ UI, CLI,     │   │ template     │   │ sync, health     │    │
│  │ Webhook      │   │ rendering)   │   │ checks)          │    │
```

```
│   └──────────────┘   └──────────────┘   └──────────────┘   │
│          │                 │                 │            │
│   ┌──────▼─────────────────▼─────────────────▼──────────┐  │
│   │                    Redis (Cache)                    │  │
│   └─────────────────────────────────────────────────────┘  │
│                                                            │
│   ┌──────────┐                                             │
│   │   Dex    │ (Optional OIDC provider for SSO)            │
│   └──────────┘                                             │
└────────────────────────────────────────────────────────────┘
          │                   │
          ▼                   ▼
     Git Repository      Kubernetes Cluster(s)
```

Component Deep Dive

3.1 API Server (`argocd-server`)

What it does:

- Exposes REST and gRPC APIs
- Serves the Web UI
- Handles CLI requests (`argocd` CLI talks to this)
- Processes webhooks from GitHub/GitLab/Bitbucket
- Enforces RBAC policies
- Manages authentication via JWT tokens or Dex OIDC

Key facts:

- Stateless — all state in Kubernetes etcd (via CRDs) and Redis cache
- Horizontally scalable
- Communicates with the Application Controller to trigger syncs

3.2 Repo Server (`argocd-repo-server`)

What it does:

- Clones Git repositories and caches them locally
- Renders manifests: processes Helm charts, runs Kustomize, renders plain YAML
- Returns the final rendered manifests to the Application Controller
- Is responsible for ALL manifest generation — never the controller

Key facts:

- ⚠ The Repo Server is the component that actually runs `helm template` or `kustomize build`
- Caches Git repos to reduce repeated cloning
- Can be scaled independently if you have many apps/repos
- Each repo is cloned once and shared across apps using the same repo

3.3 Application Controller (`argocd-application-controller`)

What it does:

- The heart of ArgoCD — runs the reconciliation loop
- Compares live cluster state vs desired state from the Repo Server
- Computes sync status: Synced / OutOfSync
- Computes health status: Healthy / Degraded / Progressing / Missing / Unknown
- Triggers sync operations when auto-sync is enabled
- Runs as a StatefulSet (not Deployment) because it maintains shard assignments

Key facts:

- ⚠ Runs as a **StatefulSet** — this is a common interview gotcha. It needs stable identity for sharding across multiple clusters
- Default reconciliation interval: **3 minutes** (configurable)
- Communicates with Repo Server to get desired manifests
- Uses Kubernetes informers to watch live cluster state

3.4 Redis

What it does:

- Caches manifests rendered by the Repo Server
- Caches cluster state queried by the Application Controller
- Acts as a pub/sub bus between components
- Reduces load on the Kubernetes API server

Key facts:

- Not used for persistent storage — all durable state is in Kubernetes CRDs
- If Redis goes down, ArgoCD degrades gracefully — it re-fetches from Git/cluster

3.5 Dex (Optional)

What it does:

- Acts as an OIDC identity provider
- Federates authentication to external providers: GitHub, Google, LDAP, SAML, Okta
- Issues OIDC tokens that ArgoCD's API server validates

Key facts:

- Required only if you want SSO. If using ArgoCD's built-in local users or your own OIDC provider, Dex is optional
- Dex is NOT required when connecting ArgoCD directly to an existing OIDC provider (e.g., Okta)

⚠ How Components Interact — The Full Flow

```
1. Developer pushes commit to Git
2. GitHub sends webhook → argocd-server
3. argocd-server notifies argocd-application-controller
```

```
4. Application Controller calls argocd-repo-server to render manifests
5. Repo Server clones/fetches repo, runs helm/kustomize, returns manifests
6. Application Controller compares rendered manifests vs live cluster
state
7. If OutOfSync AND auto-sync enabled → Application Controller applies to
cluster
8. Status (Synced/Health) stored in Application CRD → visible in UI/CLI
```

🎯 Short Interview Answer

> *"ArgoCD has three main components: the API Server which handles UI, CLI, and webhooks; the Repo Server which clones Git repos and renders manifests using Helm or Kustomize; and the Application Controller which runs the reconciliation loop comparing desired state from Git against live cluster state. Redis caches manifests and cluster data, and Dex handles SSO if needed. The Application Controller runs as a StatefulSet because it manages sharding across multiple clusters."*

Common Interview Questions

Q: Why does the Application Controller run as a StatefulSet? A: Because it uses sharding — when managing many clusters, multiple controller instances each take ownership of a subset of clusters. StatefulSet gives stable network identities and ordered deployment needed for leader election and shard assignment.

Q: What happens if the Repo Server is unavailable? A: ArgoCD can't render new manifests, so sync operations will fail. However, it can still show existing application health using cached cluster state in Redis. Existing running workloads are unaffected.

Q: What does the Application Controller use to watch cluster state? A: Kubernetes informers — the same mechanism used by native Kubernetes controllers. This is more efficient than polling the API server directly.

---

4. Core Concepts: Applications, Projects, AppSets

4.1 Application

An **Application** is the core ArgoCD CRD. It defines:

- **Source**: Where the Git repo and path are
- **Destination**: Which cluster and namespace to deploy to
- **Sync Policy**: How to handle sync (manual vs automatic)

```yaml
# Basic Application Example
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app
  namespace: argocd           # Always deployed in argocd namespace
  finalizers:
```

```yaml
    - resources-finalizer.argocd.argoproj.io  # ⚠ Enables cascade delete
  spec:
    project: default            # ArgoCD Project this app belongs to

    source:
      repoURL: https://github.com/myorg/gitops-repo.git
      targetRevision: main       # Branch, tag, or commit SHA
      path: apps/my-app          # Path inside the repo

    destination:
      server: https://kubernetes.default.svc  # In-cluster
      namespace: my-app-ns

    syncPolicy:
      automated:
        prune: true             # Delete resources removed from Git
        selfHeal: true          # Revert manual cluster changes
      syncOptions:
        - CreateNamespace=true  # Auto-create namespace if missing
        - PrunePropagationPolicy=foreground
        - ApplyOutOfSyncOnly=true  # Only apply changed resources
      retry:
        limit: 5
        backoff:
          duration: 5s
          factor: 2
          maxDuration: 3m
```

Application Status Fields

```yaml
  status:
    sync:
      status: Synced          # Synced | OutOfSync
      revision: abc123        # Current Git commit SHA
    health:
      status: Healthy         # Healthy | Degraded | Progressing | Missing |
  Unknown
    operationState:
      phase: Succeeded        # Running | Succeeded | Failed | Error
    conditions:               # List of any warning/error conditions
      - type: SyncError
        message: "..."
```

⚠ The `finalizers` Gotcha

The `resources-finalizer.argocd.argoproj.io` finalizer means: when you delete the ArgoCD Application, it also deletes all Kubernetes resources it manages. **Without this finalizer, deleting the ArgoCD App leaves the cluster resources running** — a major operational footgun.

4.2 AppProject (ArgoCD Projects)

Projects are **multi-tenancy and security boundaries** in ArgoCD. They control:

- Which Git repos an app can source from
- Which clusters an app can deploy to
- Which Kubernetes resources are allowed or denied
- Which namespaces are allowed

```yaml
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: team-backend
  namespace: argocd
spec:
  description: "Backend team project"

  # Which Git repos are allowed
  sourceRepos:
    - https://github.com/myorg/backend-*.git
    - https://github.com/myorg/shared-charts.git

  # Which clusters and namespaces are allowed
  destinations:
    - server: https://kubernetes.default.svc
      namespace: backend-*         # Glob patterns supported
    - server: https://prod-cluster.example.com
      namespace: backend-prod

  # Kubernetes resources allowed to create
  clusterResourceWhitelist:
    - group: ''
      kind: Namespace

  # Namespace-level resources blacklist (deny list)
  namespaceResourceBlacklist:
    - group: ''
      kind: ResourceQuota

  # Roles within the project
  roles:
    - name: developer
      description: "Read-only access"
      policies:
        - p, proj:team-backend:developer, applications, get, team-
backend/*, allow
      groups:
        - backend-devs        # Maps to SSO group

  # Sync windows - only allow sync during certain times
  syncWindows:
    - kind: allow
```

```yaml
        schedule: "0 8 * * 1-5"   # Mon-Fri 8am
        duration: 10h
        applications:
          - '*'
      - kind: deny
        schedule: "0 0 * * 5"    # Friday midnight
        duration: 8h
        manualSync: true          # Block even manual syncs
```

⚠ `default` Project

Every app that doesn't specify a project goes into the `default` project, which by default allows ALL repos and ALL destinations. **In production, never use the default project for real apps** — always create specific projects with restrictions.

4.3 ApplicationSet

ApplicationSets are covered in detail in Section 11. At a high level, they're a controller that **auto-generates multiple Application objects** based on generators (Git directory, cluster list, Helm values files, etc.).

🎯 Short Interview Answer — Application vs Project

> *"An Application in ArgoCD maps a Git source to a Kubernetes destination and defines how sync happens. A Project is a security boundary that controls which repos, clusters, namespaces, and resource types are allowed for a group of applications. Projects also support sync windows to control when deployments can happen. In production you'd never use the default project — you create team-specific projects with least-privilege access."*

Common Interview Questions

Q: What is the cascade delete behavior in ArgoCD? A: If the `resources-finalizer.argocd.argoproj.io` finalizer is set on the Application, deleting the ArgoCD Application also deletes all managed Kubernetes resources. Without the finalizer, only the ArgoCD Application object is deleted and cluster resources remain.

Q: How do you prevent a team from deploying to the wrong namespace? A: Use AppProject `destinations` field to whitelist specific clusters and namespaces. The Application Controller enforces that an app's destination matches its project's allowed destinations.

Q: What is a sync window? A: Sync windows define time-based rules that allow or deny sync operations. Useful for enforcing deployment freeze windows (e.g., no deployments Friday nights). They can block even manual syncs if configured to do so.

---

5. Sync Policies, Sync Waves, and Sync Hooks

5.1 Sync Policies

Manual vs Automated Sync

Mode	Behavior
Manual (default)	Git changes detected, but human must click Sync
`automated`	ArgoCD syncs automatically when it detects OutOfSync
`automated + selfHeal`	Also reverts manual cluster changes
`automated + prune`	Also deletes resources removed from Git

⚠ `prune: true` — Critical Production Decision

Without `prune: true`, resources deleted from Git remain running in the cluster. This means:

- Old Deployments, Services keep running silently
- Resource leak over time
- "Why is this old version still running?" issues

With `prune: true`: ArgoCD deletes cluster resources when they're removed from Git — which is almost always what you want, but be careful when first enabling it on an existing app.

syncOptions Deep Dive

```
syncPolicy:
  syncOptions:
    - CreateNamespace=true          # Create namespace if it doesn't exist
    - PrunePropagationPolicy=foreground  # Wait for dependent resources to
delete
    - PruneLast=true                # Prune after all other resources sync
    - ApplyOutOfSyncOnly=true       # Only apply resources that have
drifted (faster)
    - Replace=true                  # Use kubectl replace instead of apply
(for CRDs)
    - ServerSideApply=true          # Use server-side apply (better for
large objects)
    - Validate=false                # Skip schema validation (use
sparingly)
    - RespectIgnoreDifferences=true # Honor ignoreDifferences in sync
```

⚠ `Replace=true` Use Case

Some Kubernetes resources (like CRDs with immutable fields) can't be patched via `kubectl apply`. Using `Replace=true` forces `kubectl replace`, which works but is destructive — it deletes and recreates the resource. Use carefully.

5.2 Sync Waves ⚠

Sync Waves control **the order in which resources are applied** during a sync operation. Resources with lower wave numbers are applied first, and ArgoCD waits for them to become healthy before proceeding to the next wave.

```yaml
# Apply this resource in wave -1 (before wave 0 defaults)
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  annotations:
    argocd.argoproj.io/sync-wave: "-1"   # Applied first

---
# Default wave is 0 - applied second
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-app
  annotations:
    argocd.argoproj.io/sync-wave: "0"

---
# Applied after Deployment is healthy
apiVersion: v1
kind: Service
metadata:
  name: my-app-svc
  annotations:
    argocd.argoproj.io/sync-wave: "1"
```

Real Production Use Case for Sync Waves

```yaml
# Wave -1: Create namespace and CRDs first
---
kind: Namespace
metadata:
  annotations:
    argocd.argoproj.io/sync-wave: "-1"

# Wave 0: Deploy database
---
kind: StatefulSet   # PostgreSQL
metadata:
  annotations:
    argocd.argoproj.io/sync-wave: "0"

# Wave 1: Deploy backend (needs DB)
---
kind: Deployment  # Backend app
metadata:
  annotations:
    argocd.argoproj.io/sync-wave: "1"

# Wave 2: Deploy frontend (needs backend)
```

```yaml
---
kind: Deployment  # Frontend
metadata:
  annotations:
    argocd.argoproj.io/sync-wave: "2"

# Wave 3: Run smoke tests
---
kind: Job   # Post-deploy verification job
metadata:
  annotations:
    argocd.argoproj.io/sync-wave: "3"
```

⚠ Sync Wave Gotcha

ArgoCD waits for a resource to be **healthy** before moving to the next wave. If your resource doesn't have a health check (e.g., a ConfigMap), ArgoCD considers it healthy immediately. But if a Deployment is in wave 0 and is stuck in `Progressing`, wave 1 never starts. **Always ensure your wave 0 resources will actually become healthy.**

5.3 Sync Hooks

Sync Hooks are Kubernetes Jobs (or any resource) that run at specific points in the sync lifecycle.

Hook Types

Hook	When It Runs
`PreSync`	Before any resources are applied
`Sync`	During sync, alongside normal resources
`PostSync`	After all resources are Synced and Healthy
`SyncFail`	When the sync operation fails
`Skip`	ArgoCD skips this resource entirely

```yaml
# PreSync Hook — run database migration before deploying
apiVersion: batch/v1
kind: Job
metadata:
  name: db-migration
  annotations:
    argocd.argoproj.io/hook: PreSync
    argocd.argoproj.io/hook-delete-policy: HookSucceeded  # Clean up after
success
spec:
  template:
    spec:
      containers:
```

```
        - name: migrate
          image: myapp:v2.0
          command: ["python", "manage.py", "migrate"]
      restartPolicy: Never
```

```yaml
# PostSync Hook — send Slack notification after deploy
apiVersion: batch/v1
kind: Job
metadata:
  name: notify-slack
  annotations:
    argocd.argoproj.io/hook: PostSync
    argocd.argoproj.io/hook-delete-policy: HookSucceeded
spec:
  template:
    spec:
      containers:
        - name: notify
          image: curlimages/curl
          command:
            - sh
            - -c
            - |
              curl -X POST $SLACK_WEBHOOK \
                -H 'Content-type: application/json' \
                --data '{"text":"Deployment succeeded!"}'
      restartPolicy: Never
```

```yaml
# SyncFail Hook — alert on failure
apiVersion: batch/v1
kind: Job
metadata:
  name: alert-on-failure
  annotations:
    argocd.argoproj.io/hook: SyncFail
    argocd.argoproj.io/hook-delete-policy: HookFailed
spec:
  template:
    spec:
      containers:
        - name: alert
          image: curlimages/curl
          command: ["curl", "-X", "POST", "$(PAGERDUTY_URL)", "-d",
"Deployment failed!"]
      restartPolicy: Never
```

Hook Delete Policies

Policy	Behavior
`HookSucceeded`	Delete hook resource after it succeeds
`HookFailed`	Delete hook resource after it fails
`BeforeHookCreation`	Delete previous hook before creating new one (default)

⚠ Hook vs Sync Wave — Common Confusion

- **Sync Waves**: Order regular app resources relative to each other
- **Sync Hooks**: Run special one-off Jobs at lifecycle points (PreSync, PostSync)
- You can combine both: a PostSync hook can be in wave 5 to run after all wave 0-4 resources

🎯 Short Interview Answer

> *"Sync policies in ArgoCD control whether syncs happen automatically or require manual trigger, whether pruning removes resources deleted from Git, and whether self-heal reverts manual cluster changes. Sync waves are annotations on resources that control apply order — lower wave numbers apply first, and ArgoCD waits for each wave to be healthy before proceeding. Sync hooks are Kubernetes Jobs that run at lifecycle points like PreSync for DB migrations or PostSync for smoke tests."*

---

6. Health Checks and Resource Tracking

6.1 Health Status Values

Status	Meaning
`Healthy`	Resource is fully operational
`Progressing`	Resource is being updated (e.g., rolling deployment)
`Degraded`	Resource failed or is not meeting desired state
`Missing`	Resource defined in Git but doesn't exist in cluster
`Unknown`	ArgoCD can't determine health (no health check defined)
`Suspended`	Resource is intentionally paused (e.g., CronJob suspended)

6.2 Built-in Health Checks

ArgoCD has built-in health checks for core Kubernetes resources:

Resource	Health Logic
`Deployment`	Healthy when all pods are ready (`readyReplicas == replicas`)
`StatefulSet`	Healthy when `readyReplicas == replicas`
`DaemonSet`	Healthy when `numberReady == desiredNumberScheduled`

Resource	Health Logic
`Service`	Healthy if LoadBalancer has an IP (for LB type)
`Ingress`	Healthy when load balancer hostname/IP assigned
`PersistentVolumeClaim`	Healthy when status is `Bound`
`Job`	Healthy when completed successfully

6.3 Custom Health Checks

For CRDs (like a Database CRD from an operator), you need to define custom health checks using Lua scripts:

```
# In argocd-cm ConfigMap
data:
  resource.customizations.health.myorg.io_Database: |
    hs = {}
    if obj.status ~= nil then
      if obj.status.phase == "Running" then
        hs.status = "Healthy"
        hs.message = "Database is running"
      elseif obj.status.phase == "Provisioning" then
        hs.status = "Progressing"
        hs.message = "Database is being provisioned"
      else
        hs.status = "Degraded"
        hs.message = obj.status.message
      end
    end
    return hs
```

6.4 ignoreDifferences — Handling Noise

Some fields get mutated by Kubernetes controllers (e.g., `caBundle` in webhooks, `replicas` when HPA is active). ArgoCD sees these as diffs and marks apps OutOfSync. Use `ignoreDifferences` to suppress specific fields:

```
spec:
  ignoreDifferences:
    # Ignore HPA-managed replica count
    - group: apps
      kind: Deployment
      jsonPointers:
        - /spec/replicas

    # Ignore auto-injected CA bundles
    - group: admissionregistration.k8s.io
      kind: MutatingWebhookConfiguration
```

```
      jsonPointers:
        - /webhooks/0/clientConfig/caBundle

    # Ignore specific label added by another controller
    - group: ''
      kind: ServiceAccount
      jsonPointers:
        - /metadata/annotations/some-controller-annotation
```

⚠ ignoreDifferences Gotcha

`ignoreDifferences` only affects sync status comparison — it does NOT prevent ArgoCD from applying the field during a sync. If you sync, ArgoCD will still write your desired value. If the controller immediately overwrites it, you'll get a perpetual drift. Use `ManagedFields` or server-side apply to properly handle controller-managed fields.

🎯 Short Interview Answer

> *"ArgoCD tracks health at the resource and application level using built-in Lua-based health checks for standard Kubernetes resources. For CRDs, you can define custom health checks in Lua. The six health states are Healthy, Progressing, Degraded, Missing, Unknown, and Suspended. When controllers mutate fields like replica count via HPA, you use ignoreDifferences to prevent ArgoCD from constantly marking apps as OutOfSync."*

---

7. Drift Detection and Reconciliation Loops

7.1 What is Drift?

Drift is when the live cluster state differs from the desired state declared in Git.

Sources of drift:

- `kubectl edit` / `kubectl patch` by an operator
- A controller or operator mutating a resource
- Manual `kubectl scale`
- Resource deleted directly from cluster
- Namespace quota changes

7.2 How Reconciliation Works

```
Every 3 minutes (default):

1. App Controller queries Repo Server → gets rendered manifests (desired
state)
2. App Controller queries Kubernetes API → gets live resources (live
state)
3. Diff is computed (like kubectl diff)
4. If diff exists:
```

```
    - Status set to OutOfSync
    - If automated sync: trigger sync
    - If selfHeal: sync even if drift was caused by manual change
  5. If no diff:
    - Status = Synced
```

7.3 Self-Heal Behavior ⚠

```yaml
syncPolicy:
  automated:
    selfHeal: true    # This is what prevents manual changes from sticking
```

With `selfHeal: true`:

- Someone does `kubectl scale deployment my-app --replicas=10`
- ArgoCD detects drift within 3 minutes (or immediately via informer)
- ArgoCD reverts back to the replica count in Git
- **The manual change is overwritten**

Without `selfHeal`:

- Manual changes persist
- App shows as OutOfSync
- Human must trigger sync

Production recommendation: Always use `selfHeal: true` in production to enforce Git as source of truth, but make sure your team understands this — manual hotfixes to the cluster won't stick.

7.4 Changing Reconciliation Interval

```yaml
# In argocd-cm ConfigMap
data:
  timeout.reconciliation: 180s   # Default 3 minutes
```

For faster detection, use webhooks instead of polling:

- GitHub → ArgoCD webhook = near-instant detection on push

7.5 Refresh — ⚠ Commonly Asked

A **Refresh** re-fetches manifests from Git and recomputes the diff against live cluster state. It updates the sync status (`Synced` / `OutOfSync`) but **never touches the cluster**.

Happens automatically every 3 minutes (the `timeout.reconciliation` interval), or instantly via Git webhook.

There are two modes:

- **Normal refresh** — uses Repo Server's cached manifests
- **Hard refresh** — clears the cache, forces a fresh Git fetch + re-render

```
argocd app get my-app --refresh        # Normal refresh
argocd app get my-app --hard-refresh   # Bypass cache
```

Use hard refresh when you suspect stale renders — e.g., after a force push or when Helm chart dependencies changed upstream.

⚠ Refresh vs Sync in one line

> Refresh tells you *what* is wrong. Sync *fixes* it.

🎯 Short Interview Answer

> *"ArgoCD runs a reconciliation loop every 3 minutes by default — it fetches desired state from Git via the Repo Server and compares it against live cluster state using Kubernetes informers. Any difference is called drift and marks the app OutOfSync. With automated sync and selfHeal enabled, ArgoCD automatically reverts drift, including manual changes made via kubectl. For faster detection, you configure Git webhooks so ArgoCD reconciles immediately on every push."*

---

8. Manifest Sources: Helm, Kustomize, Plain YAML

8.1 Plain YAML / Directory

```yaml
spec:
  source:
    repoURL: https://github.com/myorg/gitops.git
    targetRevision: main
    path: apps/my-app/manifests   # Directory of plain YAML files

    # Optional: Only apply specific files
    directory:
      include: "*.yaml"
      exclude: "test-*.yaml"
      recurse: true               # Recurse into subdirectories
```

8.2 Helm

Option 1: Helm repo (chart lives in a Helm repo)

```yaml
spec:
  source:
    repoURL: https://charts.bitnami.com/bitnami
    chart: postgresql
```

```
    targetRevision: 12.5.6        # Helm chart version

    helm:
      releaseName: my-postgresql
      values: |
        primary:
          persistence:
            size: 10Gi
        auth:
          database: mydb
```

Option 2: Helm chart in Git repo

```
spec:
  source:
    repoURL: https://github.com/myorg/gitops.git
    targetRevision: main
    path: charts/my-app           # Path to the chart directory

    helm:
      releaseName: my-app
      valueFiles:
        - values.yaml
        - values-prod.yaml        # Environment-specific overrides
      parameters:
        - name: image.tag
          value: "v2.3.1"
      ignoreMissingValueFiles: true
```

⚠ Helm Gotcha: ArgoCD vs Helm Ownership

When ArgoCD manages a Helm-based app, it does NOT use `helm install` / `helm upgrade` under the hood. **ArgoCD renders the Helm chart to plain Kubernetes manifests and applies them directly.** This means:

- `helm ls` will NOT show your ArgoCD-managed Helm releases
- Helm hooks in the chart are NOT executed (ArgoCD has its own hook system)
- `helm rollback` won't work — rollback is done via ArgoCD/Git

8.3 Kustomize

```
spec:
  source:
    repoURL: https://github.com/myorg/gitops.git
    targetRevision: main
    path: apps/my-app/overlays/prod   # Points to kustomization.yaml

    kustomize:
```

```yaml
    namePrefix: prod-              # Add prefix to all resource names
    nameSuffix: -v2
    images:
      - name: myapp
        newTag: v2.3.1             # Override image tag
    commonLabels:
      env: production
    patches:
      - target:
          kind: Deployment
          name: my-app
        patch: |
          - op: replace
            path: /spec/replicas
            value: 5
```

Typical Kustomize Repo Structure

```
apps/my-app/
├── base/
│   ├── deployment.yaml
│   ├── service.yaml
│   └── kustomization.yaml
└── overlays/
    ├── dev/
    │   ├── kustomization.yaml
    │   └── patches/replicas.yaml
    ├── staging/
    │   └── kustomization.yaml
    └── prod/
        ├── kustomization.yaml
        └── patches/replicas.yaml
```

8.4 Multiple Sources ⚠

ArgoCD supports multiple sources per Application (alpha/beta feature):

```yaml
spec:
  sources:
    - repoURL: https://charts.bitnami.com/bitnami
      chart: postgresql
      targetRevision: 12.5.6
    - repoURL: https://github.com/myorg/gitops.git
      targetRevision: main
      path: apps/postgresql/values
      helm:
        valueFiles:
          - $values/prod-values.yaml  # Reference from second source
```

🎯 Short Interview Answer

> *"ArgoCD supports three manifest types: plain YAML directories, Helm, and Kustomize. For Helm, it's important to know that ArgoCD renders the chart to plain manifests rather than using helm install — so helm ls won't show ArgoCD-managed apps and Helm hooks don't run. For Kustomize, you typically have a base directory with overlays per environment. The Repo Server is responsible for all manifest rendering before the Application Controller applies them to the cluster."*

---

9. GitOps Repo Structure for Production

Single-Repo Structure (Small Teams)

```
gitops-repo/
├── apps/                           # Application manifests
│   ├── frontend/
│   │   ├── base/
│   │   │   ├── deployment.yaml
│   │   │   ├── service.yaml
│   │   │   └── kustomization.yaml
│   │   └── overlays/
│   │       ├── dev/
│   │       ├── staging/
│   │       └── prod/
│   ├── backend/
│   └── payments/
├── infrastructure/                 # Infra components
│   ├── cert-manager/
│   ├── ingress-nginx/
│   └── monitoring/
├── clusters/                       # Cluster-level ArgoCD apps
│   ├── dev/
│   │   └── applications.yaml       # App of Apps for dev
│   ├── staging/
│   └── prod/
└── argocd/                         # ArgoCD config itself
    ├── projects/
    ├── repositories.yaml
    └── rbac/
```

Multi-Repo Structure (Large Teams, Separation of Concerns)

```
# Repo 1: Application code (owned by dev teams)
app-source-repo/
├── src/
├── Dockerfile
└── helm/            # OR: publish chart to Helm repo
```

```
# Repo 2: GitOps config (owned by platform team)
gitops-config-repo/
├── environments/
│   ├── dev/
│   │   └── my-app/
│   │       └── values.yaml    # image.tag: latest
│   ├── staging/
│   │   └── my-app/
│   │       └── values.yaml    # image.tag: v1.2.3
│   └── prod/
│       └── my-app/
│           └── values.yaml    # image.tag: v1.2.0
├── argocd-apps/
│   ├── my-app-dev.yaml
│   ├── my-app-staging.yaml
│   └── my-app-prod.yaml
└── clusters/
```

⚠ Why Separate the Source and Config Repos?

The CI pipeline (Jenkins, GitHub Actions) **only writes to the GitOps config repo** (updating image tags). It never touches the source repo post-merge or the cluster directly. This gives:

- Clear separation between "build" (CI) and "deploy" (ArgoCD)
- Config repo can have stricter access controls
- Audit trail of exactly what version was deployed when

---

10. App of Apps Pattern

What It Is

The App of Apps pattern is using an ArgoCD Application whose manifests are **other ArgoCD Application CRDs**. This creates a hierarchy:

```
Root App (App of Apps)
├── ArgoCD App → team-backend namespace
├── ArgoCD App → team-frontend namespace
├── ArgoCD App → cert-manager
├── ArgoCD App → ingress-nginx
└── ArgoCD App → monitoring stack
```

Why It Exists

Without App of Apps, you'd manually create each Application in ArgoCD. With App of Apps, you commit Application manifests to Git and the root app deploys them declaratively. **Even ArgoCD app configuration is GitOps-managed.**

Implementation

```
gitops-repo/
├── root-app.yaml              # The "root" app of apps
└── argocd-apps/               # This directory contains Application CRDs
    ├── backend.yaml
    ├── frontend.yaml
    ├── cert-manager.yaml
    └── monitoring.yaml
```

```yaml
# root-app.yaml — manually applied once to bootstrap
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: root-app
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/myorg/gitops.git
    targetRevision: main
    path: argocd-apps             # Contains other Application YAMLs
  destination:
    server: https://kubernetes.default.svc
    namespace: argocd
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

```yaml
# argocd-apps/backend.yaml — managed by root-app
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: backend
  namespace: argocd
  finalizers:
    - resources-finalizer.argocd.argoproj.io
spec:
  project: team-backend
  source:
    repoURL: https://github.com/myorg/gitops.git
    targetRevision: main
    path: apps/backend/overlays/prod
  destination:
    server: https://kubernetes.default.svc
    namespace: backend-prod
  syncPolicy:
    automated:
```

```
      prune: true
      selfHeal: true
```

⚠ App of Apps vs ApplicationSet

	App of Apps	ApplicationSet
Use case	Bootstrapping cluster, organizing apps	Generating many similar apps (multi-cluster, multi-env)
How it works	Applications in Git	Generator + template in one CRD
Flexibility	Manual, full control	Auto-generated, template-driven
Deletion behavior	More controlled	ApplicationSet deletion can cascade

🎯 Short Interview Answer

> *"The App of Apps pattern is where one ArgoCD Application manages a directory of other Application CRDs in Git. This means even your ArgoCD application configuration is GitOps-managed — you don't need to imperatively create apps. You bootstrap one root app manually, and it deploys all other apps declaratively from Git. It's different from ApplicationSets, which are better for auto-generating many similar apps across clusters or environments."*

---

11. ApplicationSets — Fleet Management

What It Is

An ApplicationSet is a CRD managed by the ApplicationSet controller that **automatically generates ArgoCD Applications** based on a template and one or more generators.

Why It Exists

Imagine you have 50 microservices and 3 environments. That's 150 Application CRDs to manage manually. ApplicationSets let you define a template once and generate all 150 automatically.

Generator Types

1. List Generator — Static list of clusters/environments

```yaml
apiVersion: argoproj.io/v1alpha1
kind: ApplicationSet
metadata:
  name: guestbook
  namespace: argocd
spec:
  generators:
    - list:
```

```yaml
        elements:
          - cluster: dev
            url: https://dev-cluster.example.com
            namespace: guestbook-dev
          - cluster: staging
            url: https://staging-cluster.example.com
            namespace: guestbook-staging
          - cluster: prod
            url: https://prod-cluster.example.com
            namespace: guestbook-prod
  template:
    metadata:
      name: 'guestbook-{{cluster}}'
    spec:
      project: default
      source:
        repoURL: https://github.com/myorg/gitops.git
        targetRevision: main
        path: 'apps/guestbook/overlays/{{cluster}}'
      destination:
        server: '{{url}}'
        namespace: '{{namespace}}'
      syncPolicy:
        automated:
          prune: true
          selfHeal: true
```

2. Cluster Generator — All registered ArgoCD clusters

```yaml
spec:
  generators:
    - clusters:
        selector:
          matchLabels:
            environment: production   # Only prod clusters
  template:
    metadata:
      name: 'monitoring-{{name}}'    # {{name}} = cluster name
    spec:
      source:
        path: apps/monitoring
      destination:
        server: '{{server}}'         # {{server}} = cluster URL
        namespace: monitoring
```

3. Git Generator — Generate from Git directory structure

```yaml
spec:
  generators:
    - git:
        repoURL: https://github.com/myorg/gitops.git
        revision: main
        directories:
          - path: apps/*/overlays/prod   # Each matching dir = one app
  template:
    metadata:
      name: '{{path.basename}}'
    spec:
      source:
        path: '{{path}}'
      destination:
        namespace: '{{path.basename}}'
```

4. Matrix Generator — Combine two generators (e.g., all apps × all clusters)

```yaml
spec:
  generators:
    - matrix:
        generators:
          - git:
              directories:
                - path: apps/*
          - clusters:
              selector:
                matchLabels:
                  environment: prod
  template:
    metadata:
      name: '{{path.basename}}-{{name}}'
    spec:
      source:
        path: '{{path}}/overlays/prod'
      destination:
        server: '{{server}}'
        namespace: '{{path.basename}}'
```

5. SCM Provider Generator — One app per repo in a GitHub org

```yaml
spec:
  generators:
    - scmProvider:
        github:
          organization: myorg
          tokenRef:
            secretName: github-token
```

```
            key: token
        filters:
          - repositoryMatch: '^service-.*'   # Only repos starting with
service-
  template:
    metadata:
      name: '{{repository}}'
    spec:
      source:
        repoURL: '{{url}}'
        path: helm
      destination:
        namespace: '{{repository}}'
```

⚠ ApplicationSet Deletion Danger

By default, deleting an ApplicationSet **deletes all generated Applications and their managed resources**. This is the cascade effect. Use the `preserveResourcesOnDeletion` policy to prevent this:

```
spec:
  syncPolicy:
    preserveResourcesOnDeletion: true
```

🎯 Short Interview Answer

> *"ApplicationSets are a higher-level CRD that automatically generates multiple ArgoCD Applications based on generators. The List generator creates apps from a static list of clusters or environments. The Cluster generator creates one app per registered cluster. The Git generator creates one app per directory match in Git. The Matrix generator combines two generators to create a cross product — for example, all services across all clusters. It's the primary tool for fleet management at scale."*

---

12. Multi-Cluster Deployments

Registering External Clusters

```
# Login to ArgoCD
argocd login argocd.example.com

# Add external cluster (uses your current kubeconfig context)
argocd cluster add my-prod-cluster-context \
  --name prod-cluster \
  --label environment=production

# ArgoCD creates a ServiceAccount in the target cluster
# and stores the credentials as a Kubernetes Secret in the argocd
namespace
```

How ArgoCD Authenticates to External Clusters

When you run `argocd cluster add`, ArgoCD:

1. Creates a `ServiceAccount` named `argocd-manager` in `kube-system` on the target cluster
2. Creates a `ClusterRole` and `ClusterRoleBinding` with necessary permissions
3. Stores the ServiceAccount token + server URL as a Kubernetes Secret in the ArgoCD namespace

```
# The secret ArgoCD creates (simplified)
apiVersion: v1
kind: Secret
metadata:
  name: prod-cluster-secret
  namespace: argocd
  labels:
    argocd.argoproj.io/secret-type: cluster
stringData:
  name: prod-cluster
  server: https://prod-cluster.example.com
  config: |
    {
      "bearerToken": "eyJ...",
      "tlsClientConfig": {
        "caData": "..."
      }
    }
```

Deploying to Multiple Clusters

```
# Deploy same app to three clusters
---
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app-prod-us
spec:
  destination:
    server: https://prod-us.example.com
    namespace: my-app
  source:
    path: apps/my-app/overlays/prod-us

---
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: my-app-prod-eu
spec:
  destination:
    server: https://prod-eu.example.com
```

```
    namespace: my-app
  source:
    path: apps/my-app/overlays/prod-eu
```

⚠ In-Cluster vs External Cluster

For the cluster where ArgoCD itself is installed, use:

```
destination:
  server: https://kubernetes.default.svc
```

This uses the in-cluster ServiceAccount — no external credentials needed.

For external clusters, use the full API server URL.

Application Controller Sharding (at scale)

When managing many clusters, the Application Controller can be sharded:

```
# In argocd-application-controller StatefulSet
env:
  - name: ARGOCD_CONTROLLER_REPLICAS
    value: "3"    # 3 shards, each manages a subset of clusters
```

Each shard pod takes a subset of clusters. This is why it's a StatefulSet — each pod has a stable index used for shard calculation.

---

13. RBAC and SSO Integration

13.1 RBAC in ArgoCD

ArgoCD RBAC is **policy-based** using a Casbin model. Policies are defined in `argocd-rbac-cm` ConfigMap.

Default Roles

Role	Permissions
`role:readonly`	View all apps, no modifications
`role:admin`	Full access to everything

Custom RBAC Policy Syntax

```
p, <subject>, <resource>, <action>, <object>, <effect>
```

- **Subject**: User, group, or role name
- **Resource**: `applications`, `clusters`, `repositories`, `logs`, `exec`
- **Action**: `get`, `create`, `update`, `delete`, `sync`, `action`, `override`
- **Object**: `<project>/<app>` for applications
- **Effect**: `allow` or `deny`

```
# argocd-rbac-cm ConfigMap
apiVersion: v1
kind: ConfigMap
metadata:
  name: argocd-rbac-cm
  namespace: argocd
data:
  policy.default: role:readonly   # Default role for all authenticated
users

  policy.csv: |
    # Define custom role
    p, role:developer, applications, get, */*, allow
    p, role:developer, applications, sync, */*, allow
    p, role:developer, applications, create, team-backend/*, allow
    p, role:developer, applications, update, team-backend/*, allow
    p, role:developer, logs, get, */*, allow

    # Assign role to group (from SSO)
    g, backend-team, role:developer

    # Direct policy for specific user
    p, alice@example.com, applications, *, */*, allow

    # Admin group
    g, platform-team, role:admin

    # Deny exec access to all non-admins
    p, role:developer, exec, create, */*, deny
```

Project-Level RBAC (AppProject roles)

```
spec:
  roles:
    - name: ci-deployer
      description: "Role for CI pipeline to sync apps"
      policies:
        - p, proj:my-project:ci-deployer, applications, sync, my-
project/*, allow
        - p, proj:my-project:ci-deployer, applications, get, my-project/*,
allow
      jwtTokens:
```

```
        - iat: 1683000000    # Issued at - used to create service account
tokens
```

Creating JWT Token for CI

```
# Create a project token for CI pipeline
argocd proj role create-token my-project ci-deployer --token-only
# Returns: eyJ...
```

13.2 SSO Integration

Option 1: Dex with GitHub OAuth

```
# argocd-cm ConfigMap
data:
  url: https://argocd.example.com
  dex.config: |
    connectors:
      - type: github
        id: github
        name: GitHub
        config:
          clientID: $dex.github.clientID
          clientSecret: $dex.github.clientSecret
          orgs:
            - name: myorg
              teams:
                - platform-team
                - backend-team
                - frontend-team
```

Option 2: Direct OIDC (Okta, Google, Azure AD)

```
# argocd-cm ConfigMap
data:
  url: https://argocd.example.com
  oidc.config: |
    name: Okta
    issuer: https://myorg.okta.com
    clientID: abc123
    clientSecret: $oidc.clientSecret
    requestedScopes:
      - openid
      - profile
      - email
      - groups
```

```
      requestedIDTokenClaims:
        groups:
          essential: true
```

Then in RBAC:

```
data:
  policy.csv: |
    g, myorg:platform-engineers, role:admin
    g, myorg:developers, role:developer
```

⚠ RBAC Gotcha: * in Objects

```
p, role:developer, applications, sync, */*, allow
```

This means: sync any application in any project. Usually too broad. Be specific:

```
p, role:developer, applications, sync, team-backend/*, allow
```

🎯 Short Interview Answer

> *"ArgoCD RBAC uses a Casbin policy model defined in the argocd-rbac-cm ConfigMap. Policies follow the pattern: subject, resource, action, object, effect. Groups from SSO providers can be mapped to ArgoCD roles. For SSO, ArgoCD uses either Dex to federate to GitHub or LDAP, or you can point it directly to an OIDC provider like Okta. Project-level roles with JWT tokens are used for CI pipelines to trigger syncs without human credentials."*

---

14. Secrets Management with ArgoCD

⚠ The Core Problem

ArgoCD syncs everything in Git to the cluster. But you **cannot put plaintext secrets in Git**. ArgoCD itself has no native secret management — you need an external solution.

Option 1: Sealed Secrets (Bitnami)

How it works: A public key encrypts secrets → encrypted SealedSecret stored in Git → Sealed Secrets controller in cluster decrypts it using private key → creates regular Kubernetes Secret

```
# Encrypt a secret
kubeseal --cert https://sealed-secrets.example.com/v1/cert.pem \
  --format yaml < my-secret.yaml > my-sealed-secret.yaml
```

```
# Commit the sealed secret to Git — it's safe!
git add my-sealed-secret.yaml && git commit -m "Add sealed secret"
```

```yaml
# my-sealed-secret.yaml (safe to commit to Git)
apiVersion: bitnami.com/v1alpha1
kind: SealedSecret
metadata:
  name: my-secret
  namespace: my-app
spec:
  encryptedData:
    DB_PASSWORD: AgB...encrypted-base64...
    API_KEY: AgC...encrypted-base64...
```

Pros: Simple, fully GitOps compatible, no external dependencies **Cons:** Secret rotation requires re-sealing, private key loss = data loss

Option 2: External Secrets Operator (ESO)

How it works: Store secrets in AWS Secrets Manager / GCP Secret Manager / HashiCorp Vault. ESO syncs secrets from external stores into Kubernetes Secrets.

```yaml
# ExternalSecret CRD — safe to commit to Git
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: my-app-secrets
  namespace: my-app
spec:
  refreshInterval: 1h            # Re-sync every hour
  secretStoreRef:
    name: aws-secrets-manager    # Reference to SecretStore
    kind: SecretStore
  target:
    name: my-app-secret          # Name of K8s Secret to create
    creationPolicy: Owner
  data:
    - secretKey: DB_PASSWORD     # Key in K8s Secret
      remoteRef:
        key: prod/my-app         # Path in AWS Secrets Manager
        property: db_password    # Field in the secret JSON
```

```yaml
# SecretStore — defines how to connect to the secret backend
apiVersion: external-secrets.io/v1beta1
kind: SecretStore
metadata:
  name: aws-secrets-manager
```

```
    namespace: my-app
  spec:
    provider:
      aws:
        service: SecretsManager
        region: us-east-1
        auth:
          serviceAccount:
            name: external-secrets-sa   # IRSA-enabled ServiceAccount
```

Pros: Centralized secret management, rotation is automatic, works with IRSA **Cons:** External dependency, more complex setup

Option 3: HashiCorp Vault + ArgoCD Vault Plugin (AVP)

How it works: ArgoCD Vault Plugin replaces placeholder annotations in manifests with actual values from Vault at render time.

```
# manifest in Git — placeholders instead of real values
apiVersion: v1
kind: Secret
metadata:
  name: my-secret
  annotations:
    avp.kubernetes.io/path: "secret/data/my-app"   # Vault path
type: Opaque
stringData:
  DB_PASSWORD: <DB_PASSWORD>     # Placeholder
  API_KEY: <API_KEY>
```

The ArgoCD Repo Server is configured with AVP as a plugin. When rendering manifests, AVP fetches values from Vault and substitutes them.

```
# argocd-cm ConfigMap — register AVP as config management plugin
data:
  configManagementPlugins: |
    - name: argocd-vault-plugin
      generate:
        command: ["argocd-vault-plugin"]
        args: ["generate", "./"]
```

Pros: Vault as central secret store, dynamic secrets, fine-grained access **Cons:** Complex setup, Vault becomes a dependency for every deployment

Comparison

Solution	GitOps Compatible	Rotation	Complexity	Best For

Solution	GitOps Compatible	Rotation	Complexity	Best For
Sealed Secrets	✅ Fully	Manual	Low	Small teams, simple setup
External Secrets	✅ (CRDs in Git)	Automatic	Medium	AWS/GCP native workloads
Vault + AVP	✅ (templates in Git)	Dynamic	High	Enterprises with Vault

🎯 Short Interview Answer

> *"ArgoCD doesn't handle secrets natively — you can't store plaintext secrets in Git. There are three main patterns: Sealed Secrets encrypts secrets with a cluster public key so the encrypted form is safe in Git; External Secrets Operator syncs secrets from AWS Secrets Manager or Vault into Kubernetes Secrets using CRDs that are safe to commit; and the ArgoCD Vault Plugin replaces placeholder values in manifests with Vault secrets at render time. In AWS environments, External Secrets with IRSA is the most common choice."*

---

15. CI + CD Integration: Jenkins + ArgoCD

The Complete CI/CD Flow

```
┌──────────────────────────────────────────────────────────────┐
│                    CI Phase (Jenkins)                        │
│                                                              │
│  Code Push → Build → Test → Docker Build → Push to Registry  │
│                                          → Update image tag  │
│                                            in GitOps repo    │
└──────────────────────────────────────────────────────────────┘
                              │
                              ▼ (Git commit to GitOps repo)
┌──────────────────────────────────────────────────────────────┐
│                    CD Phase (ArgoCD)                         │
│                                                              │
│  Git webhook → Reconcile → Detect change → Sync to cluster   │
└──────────────────────────────────────────────────────────────┘
```

Jenkins Pipeline for GitOps

```
// Jenkinsfile — CI pipeline that updates GitOps repo
pipeline {
    agent any

    environment {
        IMAGE_REPO = 'myorg/my-app'
        GITOPS_REPO = 'https://github.com/myorg/gitops-config.git'
        ARGOCD_SERVER = 'argocd.example.com'
    }
```

```
    stages {
        stage('Build & Test') {
            steps {
                sh 'mvn clean test'
            }
        }

        stage('Docker Build & Push') {
            steps {
                script {
                    def imageTag = "${env.GIT_COMMIT[0..7]}"
                    withCredentials([usernamePassword(
                        credentialsId: 'docker-registry',
                        usernameVariable: 'DOCKER_USER',
                        passwordVariable: 'DOCKER_PASS'
                    )]) {
                        sh """
                            docker build -t ${IMAGE_REPO}:${imageTag} .
                            echo $DOCKER_PASS | docker login -u
$DOCKER_USER --password-stdin
                            docker push ${IMAGE_REPO}:${imageTag}
                        """
                    }
                    env.IMAGE_TAG = imageTag
                }
            }
        }

        stage('Update GitOps Repo') {
            steps {
                withCredentials([usernamePassword(
                    credentialsId: 'github-token',
                    usernameVariable: 'GIT_USER',
                    passwordVariable: 'GIT_TOKEN'
                )]) {
                    sh """
                        git clone
https://${GIT_USER}:${GIT_TOKEN}@github.com/myorg/gitops-config.git
                        cd gitops-config

                        # Update image tag in values file
                        sed -i 's|tag:.*|tag: ${env.IMAGE_TAG}|' \
                            environments/prod/my-app/values.yaml

                        git config user.email "jenkins@ci.com"
                        git config user.name "Jenkins CI"
                        git add .
                        git commit -m "chore: update my-app image to
${env.IMAGE_TAG}"
                        git push origin main
                    """
                }
            }
        }
```

```
        stage('Trigger ArgoCD Sync') {
            // Optional - ArgoCD auto-syncs on Git push via webhook
            // Use this only for manual sync or to wait for sync
completion
            steps {
                withCredentials([string(
                    credentialsId: 'argocd-token',
                    variable: 'ARGOCD_TOKEN'
                )]) {
                    sh """
                        argocd app sync my-app \
                            --server ${ARGOCD_SERVER} \
                            --auth-token ${ARGOCD_TOKEN} \
                            --grpc-web

                        # Wait for sync to complete
                        argocd app wait my-app \
                            --server ${ARGOCD_SERVER} \
                            --auth-token ${ARGOCD_TOKEN} \
                            --health \
                            --timeout 300
                    """
                }
            }
        }
    }

    post {
        failure {
            // Notify on failure
            slackSend(
                channel: '#deployments',
                color: 'danger',
                message: "Deploy failed: ${env.JOB_NAME} -
${env.IMAGE_TAG}"
            )
        }
    }
}
```

Using ArgoCD API Instead of CLI

```
// Trigger sync via REST API (no CLI needed in Jenkins agent)
stage('Sync via API') {
    steps {
        withCredentials([string(credentialsId: 'argocd-token', variable:
'TOKEN')]) {
            sh """
                curl -X POST \
                  https://${ARGOCD_SERVER}/api/v1/applications/my-app/sync
```

```
  \
                    -H "Authorization: Bearer ${TOKEN}" \
                    -H "Content-Type: application/json" \
                    -d '{"prune": true}'
              """
          }
      }
  }
```

⚠ ArgoCD Token for CI — Best Practice

Never use your personal ArgoCD credentials in CI. Instead:

1. Create an AppProject-scoped JWT token with minimal permissions (sync only)
2. Store it as a Jenkins credential
3. Token can have a TTL for automatic expiry

```
# Create token for CI with 90-day expiry
argocd proj role create-token my-project ci-role \
  --token-only \
  --expires-in 2160h    # 90 days
```

🎯 Short Interview Answer

> *"In a CI+CD setup with Jenkins and ArgoCD, Jenkins handles the CI phase: building, testing, and pushing Docker images. The final Jenkins step updates the image tag in the GitOps repository — it commits to the config repo, not the cluster directly. ArgoCD detects the Git change via webhook and syncs the new image to the cluster. This maintains clear separation: Jenkins owns CI, ArgoCD owns CD. For Jenkins to optionally trigger or wait for ArgoCD syncs, it uses an ArgoCD JWT token scoped to only sync permissions for the specific project."*

---

16. Argo Rollouts: Progressive Delivery

What is Argo Rollouts?

Argo Rollouts is a Kubernetes controller that replaces the standard Deployment for advanced deployment strategies:

- **Blue/Green** deployments
- **Canary** deployments with traffic splitting
- Integration with service meshes (Istio, Linkerd) and ingress controllers

⚠ ArgoCD vs Argo Rollouts — Different Tools

- **ArgoCD**: GitOps CD tool — syncs manifests to cluster
- **Argo Rollouts**: Progressive delivery controller — manages HOW pods are updated

They complement each other: ArgoCD deploys the Rollout resource, and Argo Rollouts manages the actual rolling update strategy.

16.1 Canary Deployment

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Rollout
metadata:
  name: my-app
spec:
  replicas: 10
  selector:
    matchLabels:
      app: my-app
  template:
    metadata:
      labels:
        app: my-app
    spec:
      containers:
        - name: my-app
          image: myorg/my-app:v2.0    # New version
          ports:
            - containerPort: 8080

  strategy:
    canary:
      canaryService: my-app-canary   # Service for canary pods
      stableService: my-app-stable   # Service for stable pods

      # Ingress traffic splitting
      trafficRouting:
        nginx:
          stableIngress: my-app-ingress

      steps:
        - setWeight: 10      # 10% traffic to canary
        - pause: {duration: 5m}
        - setWeight: 25      # 25% traffic
        - pause: {}          # Manual approval pause
        - setWeight: 50
        - pause: {duration: 10m}
        - analysis:          # Run analysis at 50%
            templates:
              - templateName: success-rate
        - setWeight: 100     # Full rollout

      # Auto rollback on failure
      autoPromotionEnabled: false    # Require manual promotion
```

Analysis Template (Automated Quality Gates)

```yaml
apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata:
  name: success-rate
spec:
  metrics:
    - name: success-rate
      interval: 1m
      count: 5             # Run 5 times
      successCondition: result[0] >= 0.95   # 95% success rate required
      failureLimit: 1      # Fail after 1 bad measurement
      provider:
        prometheus:
          address: http://prometheus:9090
          query: |
            sum(rate(http_requests_total{app="my-app",status!~"5.."}[5m]))
            /
            sum(rate(http_requests_total{app="my-app"}[5m]))
```

16.2 Blue/Green Deployment

```yaml
spec:
  strategy:
    blueGreen:
      activeService: my-app-active       # Production traffic
      previewService: my-app-preview     # New version (blue) traffic

      autoPromotionEnabled: false        # Manual promotion
      prePromotionAnalysis:              # Run analysis before promotion
        templates:
          - templateName: smoke-test

      postPromotionAnalysis:             # Run analysis after promotion
        templates:
          - templateName: success-rate

      scaleDownDelaySeconds: 300         # Keep old version for 5 min
after switch
```

Blue/Green Promotion Flow

```bash
# 1. Rollout creates new pods (preview/blue)
# 2. Preview service routes to new pods for testing
# 3. Manual approval:
kubectl argo rollouts promote my-app
```

```
# Or via CLI:
argocd app actions run my-app resume --kind Rollout --resource-name my-app
```

16.3 ArgoCD + Argo Rollouts Integration

ArgoCD understands Rollout health via a custom health check:

```
# argocd-cm — custom health check for Rollout
data:
  resource.customizations.health.argoproj.io_Rollout: |
    hs = {}
    if obj.status ~= nil then
      if obj.status.phase == "Paused" then
        hs.status = "Suspended"
        hs.message = "Rollout is paused"
      elseif obj.status.phase == "Progressing" then
        hs.status = "Progressing"
      elseif obj.status.phase == "Degraded" then
        hs.status = "Degraded"
        hs.message = obj.status.message
      elseif obj.status.phase == "Healthy" then
        hs.status = "Healthy"
      end
    end
    return hs
```

With this, ArgoCD shows the correct health state for Rollouts in the UI and won't mark the app Healthy until the Rollout is fully promoted.

16.4 Rollout CLI Commands

```
# Install kubectl plugin
kubectl krew install argo-rollouts

# Watch rollout progress
kubectl argo rollouts get rollout my-app --watch

# Manually promote (move to next step)
kubectl argo rollouts promote my-app

# Pause at current step
kubectl argo rollouts pause my-app

# Abort and rollback
kubectl argo rollouts abort my-app

# Set specific image (triggers new rollout)
kubectl argo rollouts set image my-app my-app=myorg/my-app:v2.1
```

🎯 Short Interview Answer

> *"Argo Rollouts is a Kubernetes controller for progressive delivery strategies like canary and blue/green. In a canary deployment, it gradually shifts traffic percentage to the new version using weighted services or ingress annotations, and can run automated analysis using Prometheus metrics to gate promotion. Blue/green deploys a new version alongside the old, and traffic switches atomically on promotion. ArgoCD manages the Rollout resource like any other Kubernetes resource — it deploys it from Git. Argo Rollouts then handles the actual update strategy. ArgoCD has built-in health check awareness for Rollout status so it doesn't mark apps healthy until the rollout is complete."*

---

17. ArgoCD CLI, UI, and API

17.1 CLI

Installation & Login

```
# Install
curl -sSL -o argocd https://github.com/argoproj/argo-
cd/releases/latest/download/argocd-linux-amd64
chmod +x argocd && mv argocd /usr/local/bin/

# Login
argocd login argocd.example.com \
  --username admin \
  --password mypassword \
  --grpc-web    # Use when behind HTTP proxy/load balancer
```

Essential Commands

```
# App management
argocd app list
argocd app get my-app
argocd app diff my-app             # Show diff between Git and cluster
argocd app sync my-app             # Trigger sync
argocd app sync my-app --dry-run   # Preview what would change
argocd app sync my-app --prune     # Also prune removed resources
argocd app wait my-app --health --timeout 120
argocd app history my-app          # Show sync history
argocd app rollback my-app 5       # Rollback to revision 5

# Cluster management
argocd cluster list
argocd cluster add <context>
argocd cluster remove https://cluster.example.com

# Repository management
```

```
argocd repo add https://github.com/myorg/gitops.git \
  --username git \
  --password ghp_...

# Project management
argocd proj list
argocd proj get my-project
argocd proj role list my-project
argocd proj role create-token my-project ci-role --token-only

# Account management
argocd account list
argocd account update-password
argocd account generate-token     # Generate token for current user
```

17.2 REST API

```
# Get auth token
TOKEN=$(curl -s https://argocd.example.com/api/v1/session \
  -d '{"username":"admin","password":"password"}' | jq -r .token)

# List applications
curl -H "Authorization: Bearer $TOKEN" \
  https://argocd.example.com/api/v1/applications

# Get specific app
curl -H "Authorization: Bearer $TOKEN" \
  https://argocd.example.com/api/v1/applications/my-app

# Trigger sync
curl -X POST \
  -H "Authorization: Bearer $TOKEN" \
  -H "Content-Type: application/json" \
  https://argocd.example.com/api/v1/applications/my-app/sync \
  -d '{
    "revision": "main",
    "prune": true,
    "dryRun": false
  }'

# Get app diff
curl -H "Authorization: Bearer $TOKEN" \
  "https://argocd.example.com/api/v1/applications/my-app/manifests"
```

17.3 Jenkins Configuration as Code with ArgoCD

```
# Bootstrap entire cluster from Git (one-time)
kubectl apply -f https://raw.githubusercontent.com/argoproj/argo-
cd/stable/manifests/install.yaml
```

```
# Then apply root app
kubectl apply -f root-app.yaml

# Everything else is managed by ArgoCD from Git
```

---

18. ArgoCD in Production: SRE Scenarios

Scenario 1: Deployment Freeze Windows

```yaml
# In AppProject — deny all syncs Friday 10pm to Monday 8am
spec:
  syncWindows:
    - kind: deny
      schedule: "0 22 * * 5"       # Friday 10pm
      duration: 58h                  # Until Monday 8am
      applications: ['*']
      namespaces: ['*']
      clusters: ['*']
      manualSync: true              # Block even manual syncs
```

Scenario 2: Automated Rollback on Health Failure

```yaml
# PostSync hook with health check — abort and rollback if unhealthy
apiVersion: batch/v1
kind: Job
metadata:
  annotations:
    argocd.argoproj.io/hook: PostSync
    argocd.argoproj.io/hook-delete-policy: HookSucceeded
spec:
  template:
    spec:
      containers:
        - name: smoke-test
          image: curlimages/curl
          command:
            - sh
            - -c
            - |
              for i in $(seq 1 10); do
                if curl -f http://my-app/health; then
                  echo "Health check passed"
                  exit 0
                fi
                sleep 10
              done
              echo "Health check failed"
```

```
                exit 1
        restartPolicy: Never
    backoffLimit: 0    # No retries — fail fast, trigger SyncFail hook
```

Scenario 3: Multi-Environment Promotion Pipeline

```
# CI updates dev, then progressive promotion via PRs
# 1. CI commits image tag to environments/dev/values.yaml
# 2. Dev sync happens automatically
# 3. Automated PR created to promote to staging
# 4. After staging validation, PR created for prod
# 5. Prod requires human approval (manual sync or PR review)
```

Scenario 4: Debugging OutOfSync Applications

```
# Step 1: See what's different
argocd app diff my-app

# Step 2: Get detailed sync status
argocd app get my-app --show-operation

# Step 3: Check events
kubectl get events -n argocd --field-selector reason=SyncError

# Step 4: Check repo server logs
kubectl logs -n argocd deployment/argocd-repo-server

# Step 5: Check app controller logs
kubectl logs -n argocd statefulset/argocd-application-controller
```

Scenario 5: ArgoCD High Availability

```
# argocd-server — scale horizontally
kubectl scale deployment argocd-server -n argocd --replicas=3

# argocd-repo-server — scale horizontally
kubectl scale deployment argocd-repo-server -n argocd --replicas=3

# argocd-application-controller — scale via sharding (StatefulSet)
kubectl scale statefulset argocd-application-controller -n argocd --
replicas=3
# Set ARGOCD_CONTROLLER_REPLICAS=3 in the StatefulSet env
```

Common Production Gotchas ⚠

Issue	Root Cause	Fix
App stuck in `Progressing`	Deployment readiness probe failing	Check pod logs, fix health
Perpetual OutOfSync	Controller mutating fields (HPA, admission webhooks)	Use `ignoreDifferences`
Sync fails on CRD	Immutable field change	Use `Replace=true` syncOption
App healthy but old version running	Auto-sync disabled, forgot to sync	Enable auto-sync or use webhooks
Deleted ArgoCD app, cluster resources gone	`finalizer` was set	Always review finalizer behavior
Helm values not applying	Wrong `valueFiles` path	Check Repo Server logs
SSO not working	Dex misconfiguration	Check Dex pod logs

---

19. Interview Quiz — Strict Mode

Instructions

Answer these out loud or in writing. Evaluate your answer against the model answers below.

Q1. What is the difference between ArgoCD's automated sync with `prune: true` vs `selfHeal: true`?

Model Answer: `prune: true` causes ArgoCD to delete Kubernetes resources that exist in the cluster but are no longer present in Git — if you remove a Deployment from your manifest, ArgoCD deletes it from the cluster. `selfHeal: true` causes ArgoCD to revert manual changes made directly to the cluster — if someone does `kubectl edit` and changes a value, ArgoCD reverts it back to what's in Git within the next reconciliation cycle. These are independent flags: you can have prune without selfHeal or vice versa.

Q2. Why does the Application Controller run as a StatefulSet rather than a Deployment?

Model Answer: The Application Controller uses sharding to distribute cluster management across multiple replicas. Each replica is responsible for a specific subset of clusters, and this assignment is based on stable pod indices. StatefulSet provides stable network identities and ordered deployment, which are required for leader election and consistent shard calculation. If it were a Deployment, pods could be replaced with new names, breaking shard assignments.

Q3. Explain what happens during a sync operation step by step.

Model Answer: When a sync is triggered: (1) The Application Controller asks the Repo Server to render manifests from Git using the configured tool (Helm/Kustomize/plain YAML). (2) The Repo Server clones or fetches the repo, renders the manifests, and returns them. (3) The Application Controller computes a diff

between the rendered manifests and the live cluster state. (4) Resources are applied in sync wave order — lowest wave number first. (5) For each wave, ArgoCD waits for all resources to become healthy before proceeding. (6) PreSync hooks run before wave 0. PostSync hooks run after all waves complete. (7) Sync status is updated to Synced or Error.

Q4. What is the difference between App of Apps and ApplicationSets? When would you use each?

Model Answer: App of Apps is a pattern where one ArgoCD Application manages a directory of other Application CRDs in Git. It's used for bootstrapping a cluster or organizing apps by team — you check in Application manifests and the root app deploys them. ApplicationSets are a CRD managed by a controller that auto-generates Applications based on generators like cluster lists, Git directories, or SCM providers. Use App of Apps when you want full manual control over each Application definition. Use ApplicationSets when you need to template and auto-generate many similar apps — for example, deploying the same service to 20 clusters, or creating one app per Git directory.

Q5. How does ArgoCD handle Helm — does it use `helm install`?

Model Answer: No. ArgoCD does NOT use `helm install` or `helm upgrade`. The Repo Server renders the Helm chart into plain Kubernetes manifests by running `helm template` internally, and those manifests are then applied directly to the cluster using `kubectl apply`. This means: `helm ls` won't show ArgoCD-managed releases, Helm hooks defined in the chart don't run (ArgoCD uses its own hook system), and `helm rollback` doesn't work — rollback is done via Git revert and ArgoCD sync.

Q6. A team is complaining that every time they `kubectl scale` their Deployment, ArgoCD reverts it. What's happening and what are the solutions?

Model Answer: This happens because `selfHeal: true` is enabled, so ArgoCD reverts manual cluster changes to match Git. There are two solutions: (1) If the team wants HPA to control replicas, add `/spec/replicas` to `ignoreDifferences` in the Application spec so ArgoCD ignores that field. Also, remove `replicas` from the Deployment manifest in Git entirely and let HPA manage it. (2) If manual scaling is needed temporarily, disable selfHeal, make the change, then re-enable it — but the proper GitOps answer is to update Git instead. The root cause is that manual scaling conflicts with GitOps — Git should always be the source of truth.

Q7. What is a Sync Wave and how does it differ from a Sync Hook?

Model Answer: A Sync Wave is an annotation on any Kubernetes resource that controls apply order within a sync operation. Resources with lower wave numbers are applied first, and ArgoCD waits for each wave to become healthy before moving to the next. A Sync Hook is a Kubernetes Job (or resource) that runs at a specific lifecycle point: PreSync before any resources apply, PostSync after all resources are healthy, or SyncFail on failure. The key difference: waves control ordering of regular app resources, hooks run special one-off tasks at lifecycle boundaries. You can combine them: a PostSync hook can itself be in a specific wave.

Q8. How would you prevent secrets from being stored in Git when using ArgoCD?

Model Answer: There are three main approaches. First, Sealed Secrets: encrypt secrets with a cluster public key using `kubeseal`, commit the SealedSecret CRD to Git (it's safe encrypted), and the Sealed Secrets controller decrypts it in-cluster. Second, External Secrets Operator: store secrets in AWS Secrets Manager or HashiCorp Vault, commit ExternalSecret CRDs to Git that define how to fetch secrets, and ESO syncs them into Kubernetes Secrets. Third, ArgoCD Vault Plugin: use placeholder values in manifests in Git, and AVP replaces them with real Vault values during the Repo Server rendering phase. In production on AWS, External Secrets with IRSA is the most common approach.

Q9. How does ArgoCD handle multi-cluster deployments? How does it authenticate to external clusters?

Model Answer: ArgoCD manages multiple clusters by storing cluster credentials as Kubernetes Secrets in the ArgoCD namespace. When you run `argocd cluster add`, ArgoCD creates an `argocd-manager` ServiceAccount in `kube-system` on the target cluster with ClusterAdmin or custom permissions, then stores the ServiceAccount token and cluster URL as a labeled Secret in the ArgoCD namespace. The Application Controller uses these credentials to apply manifests to external clusters. For the in-cluster deployment, it uses the `https://kubernetes.default.svc` address with the pod's own ServiceAccount. At scale, the Application Controller shards cluster management across multiple replicas.

Q10. What is the `resources-finalizer` on an Application and what happens if you delete an Application without it?

Model Answer: The finalizer `resources-finalizer.argocd.argoproj.io` controls cascade delete behavior. When this finalizer is present and you delete the ArgoCD Application object, ArgoCD first deletes all Kubernetes resources it manages (Deployments, Services, etc.) before removing the Application CRD. Without this finalizer, deleting the ArgoCD Application object only removes the CRD itself — all Kubernetes resources remain running in the cluster. In production, you almost always want the finalizer so orphaned resources don't accumulate when apps are decommissioned.

📊 Score Your Answers

Score	Level
8-10 correct	✅ Senior/Staff Engineer level — ready to interview
5-7 correct	🟡 Mid-level — review weak areas and practice more
3-4 correct	🔴 Junior level — revisit architecture and core concepts
0-2 correct	Study the full guide again from Section 3 onwards

---

📌 Quick Reference Cheat Sheet

ArgoCD Application States

```
Sync Status:   Synced | OutOfSync | Unknown
Health Status: Healthy | Progressing | Degraded | Missing | Unknown |
```

```
  Suspended
  Operation:     Running | Succeeded | Failed | Error | Terminating
```

Key Annotations

```
# Sync waves
argocd.argoproj.io/sync-wave: "0"

# Hooks
argocd.argoproj.io/hook: PreSync|Sync|PostSync|SyncFail|Skip
argocd.argoproj.io/hook-delete-policy:
HookSucceeded|HookFailed|BeforeHookCreation

# Managed resources (to mark a resource as managed)
argocd.argoproj.io/managed-by: my-app
```

Key CLI Commands

```
argocd app sync <app>                      # Sync app
argocd app diff <app>                      # Show diff
argocd app wait <app> --health             # Wait for healthy
argocd app rollback <app> <revision>       # Rollback
argocd app history <app>                   # Show history
argocd proj role create-token <proj> <role> --token-only  # CI token
argocd cluster add <context>               # Add cluster
```

Architecture Summary

Component	Role	Type
`argocd-server`	API, UI, CLI, webhooks	Deployment
`argocd-repo-server`	Git clone, manifest rendering	Deployment
`argocd-application-controller`	Reconcile loop, sync	**StatefulSet**
`argocd-dex-server`	SSO/OIDC federation	Deployment
`redis`	Cache layer	Deployment

Guide covers ArgoCD v2.x/v3.x — always verify against official docs at argo-cd.readthedocs.io